

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                           |
|--------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                      |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \vee ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                          |
| Sumator n-bitowy   | $n \times \text{FA}$                                                | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to Carry Out całości. |

|                     |                         |                                                                                                                     |
|---------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                     |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                      |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                          |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0         | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Bity, bajty i typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$
$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

**Skróty:** **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2). Stąd **B2U** = bity  $\rightarrow$  unsigned, podobnie **U2T**, **U**Max****, **T**Max****, **U**Add****, **T**Add****.

| Stała              | Bity          | Wartość                            |
|--------------------|---------------|------------------------------------|
| <b>U<b>Max</b></b> | <b>11...1</b> | $2^w - 1$                          |
| <b>T<b>Max</b></b> | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> )   |
| <b>T<b>Min</b></b> | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )      |
| <b>-1</b>          | <b>11...1</b> | te same bity co <b>U<b>Max</b></b> |

**Promocja w C:** **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ )  $\rightarrow$  **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow \text{U**Max**}$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                       |                                                  |
|---------------------------------------|--------------------------------------------------|
| <b>Rozszerz. 0</b> ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry              |
| <b>Rozszerz. znak</b> ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)          |
| <b>x &lt;&lt; k</b>                   | $x \cdot 2^k$ (sgn./uns.)                        |
| <b>u &gt;&gt; k</b>                   | $\lfloor u/2^k \rfloor$ — logiczne (zera)        |
| <b>x &gt;&gt; k</b>                   | arytmetyczne (kopia MSB), zaokr. do $-\infty$    |
| $x/2^k$ do 0                          | <b>(x + (1&lt;&lt;k)-1) &gt;&gt; k</b>           |
| Strength red.: <b>x*24</b>            | $\rightarrow$ <b>(x&lt;&lt;5)-(x&lt;&lt;3)</b> . |

5. Operacje, overflow i endianness

|                                        |                                                    |
|----------------------------------------|----------------------------------------------------|
| <b>UAdd</b>                            | $(u + v) \bmod 2^w$ ; carry ignorowane             |
| <b>TAdd</b>                            | bitowo = UAdd; różne znaki nie przepełniają        |
| <b>Nadmiar (+)</b>                     | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )         |
| <b>Nadmiar (-)</b>                     | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )      |
| <b>Negacja U2</b>                      | <b>-x = ~x+1</b> ; <b>-TMin=TMin</b> , <b>-0=0</b> |
| <b>Wykr. nadmiaru</b> ( <b>s=x+y</b> ) | <b>((s^x)&amp;(s^y))&gt;&gt;(w-1)</b>              |
| <b>Maska / abs</b>                     | <b>m=x&gt;&gt;31; abs=(x^m)-m</b>                  |

**Pamięć:** adres wskazuje najmłodszy bajt słowa. Napis = **char[]** ASCII + **0x00** (niezależny od endianness).

| Schemat       | Najniższy adres | <b>0x0123</b>   |
|---------------|-----------------|-----------------|
| Big Endian    | MSB             | <b>[01][23]</b> |
| Little Endian | LSB             | <b>[23][01]</b> |

## 6. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$       Bias =  $2^{k-1} - 1$

| Format          | bity | s | exp (k)        | frac (n) |
|-----------------|------|---|----------------|----------|
| Single (float)  | 32   | 1 | 8 (Bias=127)   | 23       |
| Double (double) | 64   | 1 | 11 (Bias=1023) | 52       |

Kategorie wartości

| Kategoria      | exp                              | Własności                                                                                                                                              |
|----------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $\neq 0\dots0$<br>$\neq 1\dots1$ | $E = \text{exp} - \text{Bias}$ . Ukryta jedynka: $M = 1.\text{frac}$ . Najgęściej ułożone wokół 0.                                                     |
| Zdenormaliz.   | $= 0\dots0$                      | $E = 1 - \text{Bias}$ . Brak jedynki: $M = 0.\text{frac}$ . Reprezentują +0.0 i -0.0 (frac = 0) oraz liczby bardzo bliskie zera (stopniowy underflow). |
| Specjalne      | $= 1\dots1$                      | Gdy frac == 0, to $+\infty$ lub $-\infty$ (nadmiar/dzielenie przez 0). Gdy frac $\neq$ 0, to NaN (np. $\sqrt{-1}$ , $\infty - \infty$ ).               |

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

| ... x x G    !    R S S S ...                               |        |                                             |
|-------------------------------------------------------------|--------|---------------------------------------------|
| G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty |        |                                             |
| Warunek                                                     | Ułamek | Akcja                                       |
| R=0                                                         | < 0.5  | W dół (odcięcie)                            |
| R=1, S=1                                                    | > 0.5  | W górę (+1 do G)                            |
| R=1, S=0                                                    | = 0.5  | Do parzystej: w górę gdy G=1, w dół gdy G=0 |

Własności matematyczne i rzutowanie

- Brak łączności:**  $(a + b) + c \neq a + (b + c)$ , przez zaokrąglenia i utratę danych.
- Brak rozdzielności:**  $a(b + c) \neq ab + ac$ .
- Rzutowanie int → float:** Może stracić precyzję (zaokrągla zgodnie z trybem).
- Rzutowanie int → double:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- Rzutowanie float/double → int:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca Tmin (0x80000000).

## 7. Rejestry x86\_64

|      |      |                |      |      |                |
|------|------|----------------|------|------|----------------|
| %rax | %eax | %ax<br>%ah %al | %rbx | %ebx | %bx<br>%bh %bl |
| %rcx | %ecx | %cx<br>%ch %cl | %rdx | %edx | %dx<br>%dh %dl |
| %rsi | %esi | %si<br>%sil    | %rdi | %edi | %di<br>%dil    |
| %rbp | %ebp | %bp<br>%bpl    | %rsp | %esp | %sp<br>%spl    |

|     |      |              |     |      |              |
|-----|------|--------------|-----|------|--------------|
| %r8 | %r8d | %r8w<br>%r8b | %r9 | %r9d | %r9w<br>%r9b |
|-----|------|--------------|-----|------|--------------|

Uwaga: Rejestry od %r10 do %r15 używają schematu: %r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

|                                  |                                                                                |
|----------------------------------|--------------------------------------------------------------------------------|
| %rax                             | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| %rdi, %rsi, %rdx, %rcx, %r8, %r9 | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| %rsp                             | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| %rbp                             | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| %rbx, %r12-%r15                  | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| %rip                             | Wskaźnik instrukcji (Program Counter).                                         |

## 8. Tryby adresowania (operandsy)

| Tryb                   | Format         | Obliczanie adresu    |
|------------------------|----------------|----------------------|
| Natychmiastowy (Imm)   | \$Imm          | Imm                  |
| Rejestrowy (Reg)       | %Ra            | %Ra                  |
| Bezpośredni (Mem)      | Imm            | M[Imm]               |
| Pośredni (Mem)         | (%Rb)          | M[%Rb]               |
| Z przesunięciem        | D(%Rb)         | M[%Rb + D]           |
| Skalowany (Ind/scaled) | D(%Rb, %Ri, S) | M[%Rb + %Ri * S + D] |
| Skalowany (baseless)   | (, %Ri, S)     | M[%Ri * S]           |

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

## 9. Katalog instrukcji (AT&T)

| Opcode         | Efekt                         | Opis                                                    |
|----------------|-------------------------------|---------------------------------------------------------|
| mov S, D       | $D \leftarrow S$              | Kopiuje wartość z S do D.                               |
| lea S, D       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                 |
| add S, D       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                              |
| sub S, D       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                            |
| imul S, D      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                              |
| sal / shl k, D | $D \ll= k$                    | Przesunięcie w lewo (mnożenie przez $2^k$ ).            |
| sar k, D       | $D \gg= k$                    | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak</b> . |
| shr k, D       | $D \gg= k$                    | Logiczne w prawo. Dopełnia zerami.                      |

| Opcode                     | Efekt                     | Opis                                                          |
|----------------------------|---------------------------|---------------------------------------------------------------|
| <code>and / or S, D</code> | $D \leftarrow D \& /   S$ | Operacje bitowe (ustawiają flagi).                            |
| <code>xor S, D</code>      | $D \leftarrow D \wedge S$ | Często używane jako <code>xor %rax, %rax</code> do zerowania. |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).